Práctica PSP · Procesos e Hilos

Antes de empezar

El proyecto

Clase	¿La tocas?	Parte
`VentanaProcesos`	✗ NO (ya hecha)	—
`ResultadoProceso`	✗ NO (ya hecha)	—
`Log`	✓ Sí	1 y 2
`GestorProcesos`	✓ Sí	1
`GestorHilos`	✓ Sí	2
`HiloProceso`	✓ Sí	2
`LectorSalida` · `LectorErrores` · `PintorSalida`	✓ Sí	2
`LanzadorMultiple` · `TareaProceso`	✓ Sí	2

- Ejecuta siempre `VentanaProcesos` (tiene el `main`).
- Las firmas de los métodos **no se cambian**: la ventana las usa.

Swing

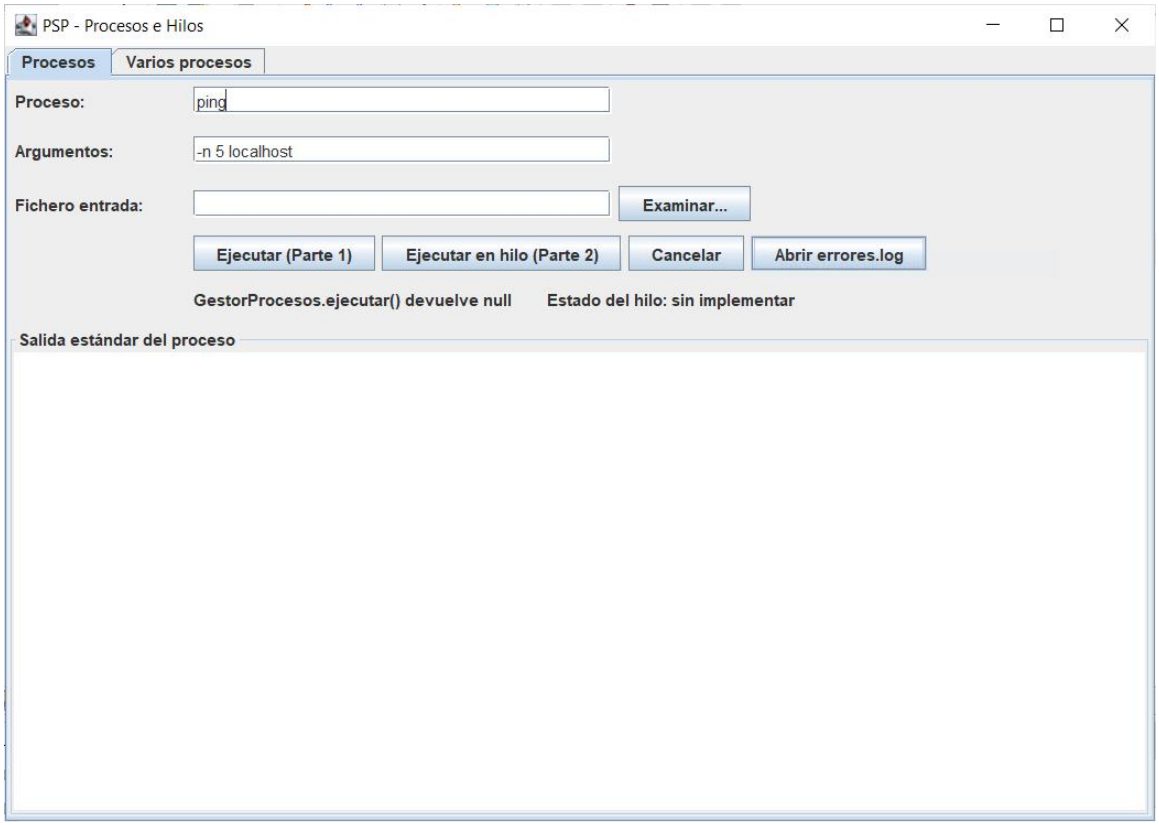
- Los componentes de Swing **solo** se pueden tocar desde el hilo de la ventana.
- Tus hilos **nunca** hacen `label.setText(...)`.
- Tus hilos usan **solo** estos métodos de `VentanaProcesos` (ya son seguros):

Método	Pestaña	Qué hace
`anadirSalida(String linea)`	Procesos	Añade una línea a la salida
`mostrarFin(String texto)`	Procesos	Cambia la etiqueta “Código de salida”
`anadirActividad(String texto)`	Varios procesos	Añade una línea a “Actividad”
`actualizarContadores(int lanzados, int enEjecucion, int terminados, int conError)`	Varios procesos	Actualiza los contadores
`todosTerminados()`	Varios procesos	Muestra “TODOS LOS PROCESOS HAN

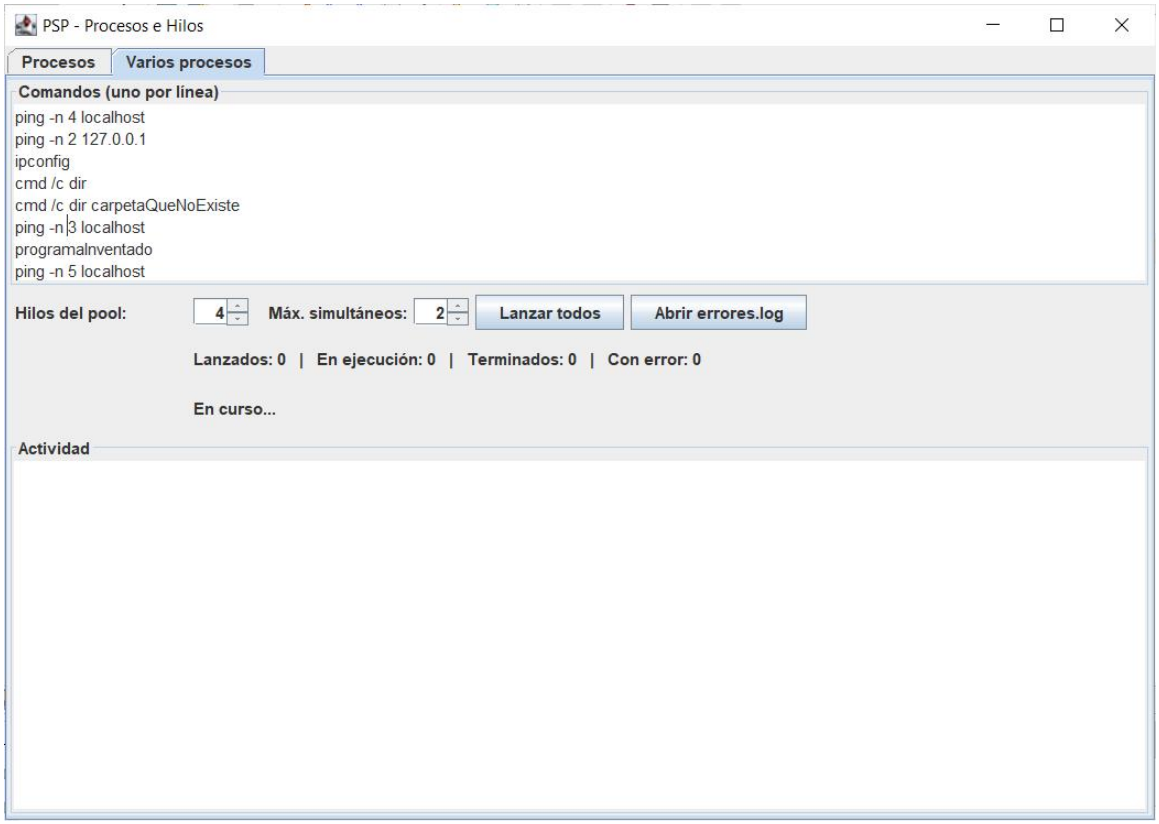
Método	Pestaña	Qué hace
		TERMINADO”

Pantallas

Parte 1 (Procesos)



Parte 2 (Hilos)



PARTE 1 · PROCESOS

Pestaña **Procesos** → botón **Ejecutar (Parte 1)**.



P1.1 · `Log.registrar(String texto)`

Escribir una línea en *errores.log* **sin borrar** lo anterior.

Implementar:

1. Monta la línea: `[fecha y hora] [nombre del hilo] texto` + salto de línea.
   - Hora: `LocalDateTime.now()` + `DateTimeFormatter`.
   - Hilo: `Thread.currentThread().getName()`.
2. Escríbela con: `Files.writeString(FICHERO.toPath(), linea, StandardCharsets.UTF_8, CREATE, APPEND)`
3. Captura la `IOException` → Si la hay, muéstrala con `System.err.println`.

P1.2 · `GestorProcesos.ejecutar(comando, argumentos, ficheroEntrada)`

Lanzar un proceso y gestionar sus **3 flujos**:

Flujo	Destino
stdin (entrada)	el fichero, si se indica
stdout (salida)	lo lees tú → se muestra en la ventana
stderr (error)	`errores.log`

Implementar:

1. Crea una `List<String>` con el comando + los argumentos → `new ProcessBuilder(lista)`.
2. Si `ficheroEntrada != null` → `pb.redirectInput(ficheroEntrada)`.
3. `Log.registrar("=== Lanzado: " + ...)`.
4. stderr al log sin borrar → `pb.redirectError(ProcessBuilder.Redirect.appendTo(Log.FICHERO))`.
5. `pb.start()`.
   - Si lanza `IOException` → regístrala en el log y **vuelve a lanzarla** (`throw e`).
6. Cierra la entrada del proceso: `proceso.getOutputStream().close()`.

7. Crea una constante **pública**: `public static final Charset CHARSET_CONSOLA = Charset.forName("CP850");` (la consola de Windows no usa UTF-8).
8. Lee stdout línea a línea: `BufferedReader` + `InputStreamReader(proceso.getInputStream(), CHARSET_CONSOLA)`. Guarda las líneas en un `StringBuilder`.
9. `int codigo = proceso.waitFor();` → regístralo en el log.
10. `return new ResultadoProceso(salida, codigo);`

Para comprobar si está bien hecho todo haz estas pruebas:

Proceso	Argumentos	Fichero	Debe pasar
`cmd`	`/c dir`	—	Listado · código 0
`ipconfig`	—	—	Configuración de red
`ping`	`-n 3 localhost`	—	3 respuestas
`sort`	—	`entrada.txt`	Líneas ordenadas
`findstr`	`mundo`	`entrada.txt`	Solo líneas con "mundo"
`cmd`	`/c dir carpetaQueNoExiste`	—	Error en `errores.log` · código ≠ 0
`programaInventado`	—	—	Error en la ventana y en el log

A revisar (para examen):

1. ¿Por qué `dir` necesita `cmd /c` y `ping` no?
2. ¿Qué pasa con `sort` sin fichero si **no** haces el paso 6?
3. Lanza `ping -n 10 localhost`. ¿Qué le pasa a la ventana? ¿Por qué? → **Esto lo arreglas en la Parte 2.**

PARTE 2 · HILOS

Bloque A · Un proceso sin congelar la ventana

Pestaña **Procesos** → botones **Ejecutar en hilo (Parte 2)** y **Cancelar**.

P2.1 · Proceso en un hilo

Necesitamos que la ventana **no se congele**.

Haz:

1. `HiloProceso` (implementa `Runnable`):
   - Guarda los 4 parámetros del constructor en atributos.
   - `run()` → **versión 1** (sencilla):
     - Llama a `GestorProcesos.ejecutar(...)`.
     - `ventana.anadirSalida(resultado.salida())`.
     - `ventana.mostrarFin("Código de salida: " + ...)`.
     - Si hay excepción → `ventana.mostrarFin("ERROR: " + mensaje)`.
2. `GestorHilos.lanzarEnHilo(...)`:
   - `new Thread(new HiloProceso(...), "HiloProceso")`.
   - `start()`.
   - Devuelve el hilo.

Para ver si está bien implementado, comprueba que:
`ping -n 10 localhost` → la ventana **se puede mover** mientras se ejecuta.

Responde (para el examen): ¿Qué pasaría si llamas a `run()` en lugar de `start()`?

P2.2 · Estado del hilo

Podemos ver los estados de un hilo en directo.

Implementa: `GestorHilos.estado(Thread hilo)`:

- Si `hilo == null` → devuelve `"sin lanzar"`.
- Si no → devuelve `hilo.getState()` + `hilo.isAlive()`.
- (La ventana lo llama sola cada 200 ms).

Para ver si está bien implementado, comprueba que:
lanza `ping -n 5 localhost` → tienes que ver `RUNNABLE` … `TERMINATED`.

Responde (para examen): ¿Por qué no ves nunca el estado `NEW`?

P2.3 · Leer los flujos con hilos (productor-consumidor)

Queremos ver la salida **línea a línea, en directo**, con 3 hilos colaborando:

```
proceso -stdout-> LectorSalida -> [cola] -> PintorSalida -> ventana
proceso -stderr-> LectorErrores -> errores.log
```

Implementa:

1. `LectorErrores` (Runnable):
   - Lee el flujo línea a línea (con `CHARSET_CONSOLA`).
   - Cada línea → `Log.registrar("[stderr] " + linea)`.
2. `LectorSalida` (**productor**):
   - Lee el flujo línea a línea.
   - Cada línea → `cola.put(linea)`.
   - Al acabar (en un `finally`) → `cola.offer(FIN)` = "ya no hay más".
3. `PintorSalida` (**consumidor**):
   - Bucle: `String linea = cola.take();`
   - Si `linea.equals(LectorSalida.FIN)` → `break`.
   - Si no → `ventana.anadirSalida(linea)`.
4. `HiloProceso.run()` → **versión 2** (ya **no** usa `GestorProcesos`):
   1. Crea el `ProcessBuilder` (comando + argumentos + fichero de entrada). **Sin** `redirectError`.
   2. `Log.registrar(...)` → `start()` → cierra `getOutputStream()`.
      - Si falla → `ventana.mostrarFin("ERROR: ...")` y `return`.
   3. `BlockingQueue<String> cola = new LinkedBlockingQueue<>();`
   4. Crea y arranca 3 hilos: `LectorSalida`, `PintorSalida`, `LectorErrores`.
   5. `int codigo = proceso.waitFor();`
   6. `join()` de los 3 hilos.
   7. Registra el código en el log → `ventana.mostrarFin(...)`.

Para ver si está bien implementado, comprueba que: :

- `ping -n 5 localhost` → las respuestas aparecen **una a una**.
- `cmd /c dir carpetaQueNoExiste` → el error aparece en el log con `[LectorErrores]`.
- Mira el estado del hilo: ahora sale `WAITING`.

Responde(para examen):

1. Antes (P2.2) salía `RUNNABLE` y ahora `WAITING`. ¿Por qué? (pista: ¿qué método está ejecutando `HiloProceso` mientras espera?)
2. ¿Por qué hay que leer stdout y stderr **a la vez**? (pista: ¿qué pasa si un proceso llena el búfer de stderr y nadie lo lee?)

3. ¿Para qué sirve `FIN`? ¿Qué pasaría sin él?

P2.4 · Cancelar

Queremos que el botón **Cancelar** detenga el proceso.

Implementa:

1. `GestorHilos.cancelar(Thread hilo)`:
   - Si `hilo != null` y está vivo → `hilo.interrupt()`.
2. `HiloProceso.run()` → en el `catch (InterruptedException e)` de `waitFor()` / `join()`:
   - `proceso.descendants().forEach(ProcessHandle::destroy);` (mata los hijos, p. ej. `cmd /c ping`).
   - `proceso.destroy();`
   - `Log.registrar("Proceso cancelado ...")`.
   - `ventana.mostrarFin("CANCELADO")`.

Para ver si está bien implementado, comprueba que:

- `ping -n 30 localhost` → **Cancelar** → sale `CANCELADO` y el estado pasa a `TERMINATED`.
- Lo mismo con `cmd` + `/c ping -n 30 localhost`.

Responde (para el examen):
¿Habría funcionado **Cancelar** con la versión 1 de P2.1? ¿Por qué? (pista: ¿`readLine()` responde a `interrupt()`?)

Bloque B · Varios procesos a la vez

Pestaña **Varios procesos** → botón **Lanzar todos**.

P2.5 · Pool de hilos

Queremos lanzar muchos comandos usando un número fijo de hilos (en vez de crear/destruir hilos en cada ejecución).

Implementa:

1. `LanzadorMultiple`:
   - Guarda la ventana en un atributo → `getVentana()` la devuelve.
   - `lanzarTodos(...)`:
     - `ExecutorService pool = Executors.newFixedThreadPool(hilosPool);`
     - Por cada línea → `pool.submit(new TareaProceso(linea, this));`
     - `pool.shutdown();`
2. `TareaProceso` (Runnable):
   - Guarda la línea y el lanzador.
   - `run()`:
     - Separa la línea: `split("\\s+")` → la primera palabra es el comando, el resto los argumentos (`Arrays.copyOfRange`).
     - `ventana.anadirActividad(">> [hilo] EMPIEZA: " + linea)`.
     - `GestorProcesos.ejecutar(comando, argumentos, null)`.
     - `ventana.anadirActividad("<< [hilo] TERMINA: " + linea + " -> código " + ...)`.
     - Si hay excepción → `ventana.anadirActividad("XX [hilo] ERROR: ...")`.

Para ver si está bien implementado, comprueba que: :

- Hilos del pool = **1** → los comandos van **de uno en uno**.
- Hilos del pool = **4** → van **de 4 en 4**. Fíjate en los nombres `pool-1-thread-N`.

Responde (para el examen): ¿Qué ventaja tiene un pool frente a crear un `new Thread` por comando?

P2.6 · Semáforo

Vamos a limitar con un semáforo cuántos procesos se ejecutan **a la vez**, aunque el pool tenga más hilos.

Implementa:

1. `LanzadorMultiple.lanzarTodos(...)` → `semaforo = new Semaphore(maxSimultaneos);` + `getSemaforo()`.
2. `TareaProceso.run()`:
   - Al principio → `anadirActividad(".. [hilo] esperando plaza: ...")` → `getSemaforo().acquire()`.
   - En el `finally`, **en este orden**:
     1. `procesoTerminado(correcto)`
     2. `getSemaforo().release()`

Para ver si está bien implementado, comprueba que:
Pool = **4** · Máx. simultáneos = **2** → **En ejecución** nunca pasa de **2**.

Responde (para el examne): ¿Por qué `release()` va en un `finally`?